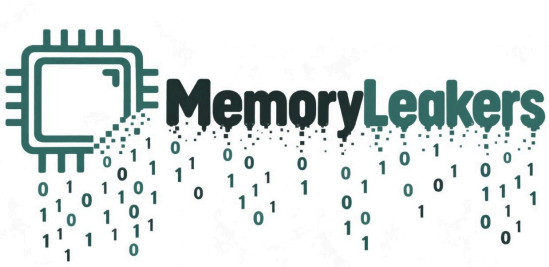
Analisi dei Capitolati

Valutazione e scelta dei capitolati d'appalto per l'A.A. 2026/2027

Gruppo: **Memory Leakers**





CORSO DI INGEGNERIA DEL SOFTWARE

Anno Accademico 2026/2027

Controllo Documentale

Titolo Doc.:	Analisi dei Capitolati
Versione:	1.0.0
Autore:	Matteo Arcandi, Jacopo Franco, Riccardo Peresson
Stato:	Approvato
Verificato da:	Chiara Protopapa
In data:	29/10/2026
Tipo documento:	Analisi dei Capitolati
File:	analisi-dei-capitolati.pdf

Registro delle Modifiche

Ver.	Data	Descrizione e/o Modifiche	Autore	Verificatore
1.0.0	30/10/2026	Firma ed approvazione del documento.	Matteo Arcandi, Jacopo Franco, Riccardo Peresson	Paolo Prevedello Ramirez
0.1.1	30/10/2026	Aggiunta link al glossario.	Riccardo Peresson	Chiara Protopapa
0.1.0	29/10/2026	Correzione imperfezioni.	Matteo Arcandi	Chiara Protopapa
0.0.2	29/10/2026	Correzione e fine stesura.	Jacopo Franco	
0.0.1	28/10/2026	Stesura iniziale del documento di analisi dei capitolati.	Matteo Arcandi	

Indice dei Contenuti

Introduzione 4
Riepilogo 4
C0 - AIRA (Sanmarco Informatica) 5
Descrizione 5
Obiettivo 5
Tecnologie richieste 5
Pro e contro 6
Pro 6
Contro 6
Conclusioni 6
C1 - AURA (Vimar) 6
Descrizione 6
Obiettivo 6

Tecnologie richieste . . . 7
Pro e contro . . . 7
Pro . . . 7
Contro . . . 7
Conclusioni . . . 7
C2 - NEXUM Concierge (Eggon) . . . 8
Descrizione . . . 8
Obiettivo . . . 8
Tecnologie richieste . . . 8
Pro e contro . . . 8
Pro . . . 8
Contro . . . 8
Conclusioni . . . 9
C3 - FossDot (Uranion) . . . 9
Descrizione . . . 9
Obiettivo . . . 9
Tecnologie richieste . . . 9
Pro e contro . . . 10
Pro . . . 10
Contro . . . 10
Conclusioni . . . 10
C4 - Piattaforma per veicoli e servizi post-vendita (M31 S.r.l.) . . . 10
Descrizione . . . 10
Obiettivo . . . 10
Tecnologie richieste . . . 10
Pro e contro . . . 11
Pro . . . 11
Contro . . . 11
Conclusioni . . . 11
C5 - RareCare Navigator (Miriade) . . . 11
Descrizione . . . 11
Obiettivo . . . 11
Tecnologie richieste . . . 12
Pro e contro . . . 12
Pro . . . 12
Contro . . . 12
Conclusioni . . . 12
C6 - Sistema AI di caching semantico e dashboard FinOps (Var Group) . . . 13
Descrizione . . . 13
Obiettivo . . . 13
Tecnologie richieste . . . 13
Rischi e punti aperti . . . 13
Motivazione della scelta . . . 14
Conclusioni . . . 14
C7 - SmartDashboard (Ergon Informatica) . . . 14
Descrizione . . . 14
Obiettivo . . . 14
Tecnologie richieste . . . 15
Pro e contro . . . 15
Pro . . . 15

Contro . . . 15
Conclusioni . . . 15
C8 - Sonar (Halue) . . . 16
Descrizione . . . 16
Obiettivo . . . 16
Tecnologie richieste . . . 16
Pro e contro . . . 16
Pro . . . 16
Contro . . . 17
Conclusioni . . . 17
C9 - SpecCheck (Zucchetti) . . . 17
Descrizione . . . 17
Obiettivo . . . 17
Tecnologie richieste . . . 17
Pro e contro . . . 18
Pro . . . 18
Contro . . . 18
Conclusioni . . . 18
Appendice A - Contesto e vincoli del gruppo . . . 18
Appendice B - Incontri esplorativi . . . 18

Introduzione

Il gruppo Memory Leakers (gruppo 3), composto da sette persone, ha analizzato tutti i capitolati proposti per l'anno accademico 2026/2027 per individuare il più affine alla candidatura; per ciascun capitolato$_G$ il documento riporta, con lo stesso schema: descrizione, obiettivo, tecnologie richieste, pro e contro, motivazione per il capitolato scelto e conclusioni del gruppo.

Fonti. Ogni valutazione si basa sulla documentazione fornita da ogni proponente$_G$ e sulle slide presentate in aula il 13/10/2026; per i capitolati C1, C5, C6 e C7 si aggiungono, inoltre, gli incontri con le aziende (Appendice B), di cui sono riportati i link ai verbali.

Criteri di valutazione. Il gruppo ha considerato: rischio tecnico e fattibilità nelle ore a disposizione, chiarezza dei requisiti e dei criteri di accettazione, supporto offerto dal proponente, interesse del gruppo e competenze di partenza, valore formativo, allineamento con i criteri del corso (documentazione, verifica, tracciabilità).

Esito. Il gruppo si candida al capitolato C6 (Var Group); in caso di mancata aggiudicazione indica come seconda scelta C7 (Ergon) e come alternative C1 (Vimar) e C5 (Miriade). Gli altri capitolati non sono stati selezionati per le ragioni riportate nelle rispettive conclusioni.

Riepilogo

Capitolato	Proponente	Incontro	Esito del gruppo
C0 AIRA	Sanmarco Informatica	no	Non selezionato: nessuna soglia di accettazione, ampiezza tecnica eccessiva
C1 AURA	Vimar	22/10/2026	Alternativa: requisiti molto chiari, tecnologie nuove e dominio meno vicino al gruppo

C2 NEXUM Concierge	Eggon	no	Non selezionato: stack AI da definire
C3 FossDot	Uranion	no	Non selezionato: hardware, firmware e backend, rischio tecnico troppo alto
C4 Veicoli e post-vendita	M31	no	Non selezionato: dominio ampio e microservizi$_G$, rischio sui tempi di analisi
C5 RareCare Navigator	Miriade	26/10/2026	Alternativa: forte valore sociale, perimetro ampio e tecnologie nuove
C6 Caching semantico e FinOps	Var Group	26/10/2026	Capitolato scelto
C7 SmartDashboard	Ergon Informatica	22/10/2026	Seconda scelta: percorso completo, perimetro ampio
C8 Sonar	Halue	no	Non selezionato: stack vincolante
C9 SpecCheck	Zucchetti	no	Non selezionato: esito incerto, nessun criterio di accettazione

C0 - AIRA (Sanmarco Informatica)

Descrizione

Sanmarco Informatica S.p.A. propone AIRA (AI Requests Automation), un sistema per gestire richieste aziendali non strutturate (ordini d'acquisto, ticket di assistenza, prenotazioni, opportunità commerciali) provenienti da canali eterogenei come e-mail, messaggistica e cartelle condivise.

Le richieste arrivano come testo, note vocali, immagini e PDF scansionati, spesso con abbreviazioni e codici incompleti: presidiare a mano ogni fonte sovraccarica gli operatori e causa errori e ritardi. Il sistema ha l'obiettivo di elaborare i diversi formati con modelli AI, OCR e speech-to-text, trasformarli in dati strutturati (ad esempio JSON$_G$) e inoltrarli ai gestionali aziendali tramite API$_G$ REST.

Obiettivo

Il capitolato richiede una pipeline che:

- monitora in continuo i canali configurati (caselle di posta, WhatsApp, cartelle su filesystem), rilevando mittente, testo e allegati;
- filtra i dati in ingresso per isolare i clienti (tenant) ed evitare accessi non autorizzati;
- converte l'audio in testo e legge immagini e documenti scansionati (OCR e visione artificiale);
- interpreta le richieste con un LLM$_G$ e le mappa su template per tipologia (ordine, ticket, caso, evento), associando ad esempio le descrizioni informali ai codici articolo del catalogo;
- avvia azioni configurabili: chiamate REST a portali B2B, ERP$_G$, CRM, ticketing oppure invio di e-mail;
- espone un'interfaccia web di amministrazione per configurare i canali, consultare lo storico dei log e gestire la persistenza dei dati.

Sono opzionali una dashboard$_G$ analitica dei flussi e le risposte automatiche al mittente.

Tecnologie richieste

Il proponente consiglia:

- Java con Spring Boot (pipeline modulare a eventi);
- LangChain4j per l'interpretazione con LLM;
- Whisper, Google Speech, Tesseract o EasyOCR per speech-to-text e OCR;
- SQLite, MariaDB o PostgreSQL per i dati;
- Angular con TypeScript per l'interfaccia amministrativa;
- configurazione esterna in JSON o YAML.

Pro e contro

Pro

- Problema aziendale concreto e ben motivato, con un'architettura$_G$ a pipeline modulare facile da suddividere fra i componenti del gruppo.
- Tecnologie accessibili e documentate; supporto con metodologia Agile$_G$ da parte di Sanmarco, azienda con solida esperienza industriale.
- Buon valore formativo: elaborazione multimodale, uso degli LLM per strutturare i dati, integrazione con servizi REST.

Contro

- Nessuna soglia quantitativa (volumi, accuratezza, copertura dei test) né criteri di accettazione misurabili.
- Ampiezza tecnica: tre tipi di elaborazione (audio, immagini, testo) più l'integrazione con i gestionali da realizzare e valutare nelle ore disponibili, senza metriche di accuratezza indicate dal proponente.
- La parte più delicata, associare descrizioni informali ai codici di catalogo, dipende da un catalogo di prova e da un livello di accuratezza non definiti; non sono forniti lo schema delle API di destinazione né dettagli sull'integrazione con WhatsApp.
- Trattamento di voce, immagini e dati commerciali potenzialmente sensibili, con isolamento dei tenant da progettare.
- Interesse limitato del gruppo per lo stack Java/Spring Boot e per il dominio B2B.

Conclusioni

Il gruppo preferisce non procedere alla candidatura per questo capitolato. L'assenza di criteri di accettazione misurabili e dei dati necessari alla parte più critica rende difficile dimostrare in modo oggettivo il valore di un MVP$_G$, e l'ampiezza tecnica (audio, immagini, testo e integrazione con i gestionali) richiede un carico elevato rispetto alle ore a disposizione. La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

C1 - AURA (Vimar)

Descrizione

Vimar S.p.A. vuole mostrare come i servizi di AI generativa esistenti (Claude, ChatGPT, Gemini, Copilot) possano essere usati come interfaccia in linguaggio naturale per gli impianti domotici residenziali Vimar View Wireless, senza sviluppare un assistente dedicato. L'impianto consiste in una rete Bluetooth Mesh collegata al cloud Vimar tramite un gateway BLE. L'accesso programmatico passa dalle KNX IoT 3rd-party API, un'interfaccia REST in cui un ambiente (*Location*) contiene funzionalità (*Function*) i cui attributi sono i *Datapoint*. Il progetto$_G$ rende queste API utilizzabili da un modello linguistico attraverso il Model Context Protocol (MCP$_G$): un server espone risorse e strumenti, un client permette al modello di scoprirli e usarli.

Obiettivo

Il capitolato richiede tre componenti obbligatori:

- **Infrastruttura cloud-ready:** container$_G$, Infrastructure as Code, avvio con un solo comando (docker compose up); compatibilità con AWS$_G$, mentre il rilascio su AWS è opzionale.

- **Server e client MCP:** il server supporta le API KNX IoT (Authorization, Datapoint, Function, Information, Location, Subscription, Timeseries), in lettura e scrittura, con autenticazione OAuth2$_G$ e ricostruzione delle relazioni tra ambienti, funzionalità e datapoint; il client si interfaccia solo con il server realizzato.
- **Agente o modello AI esterno commerciale:** con autenticazione dell'utente tramite account MyVimar, scelta e memorizzazione dell'impianto nella conversazione, lettura e modifica dello stato dei dispositivi.

I casi d'uso previsti sono 17 casi base (accesso, elenco impianti, lettura e comando di luci e tapparelle, comandi su più dispositivi) e 21 casi avanzati (richieste ambigue, sinonimi, richieste implicite come «è buio», conferma per comandi estesi, elementi non trovati). Sono *nice-to-have* gli scenari multi-impianto, i comandi condizionali, lo storico e i trigger. La copertura dei test deve essere almeno del 75% su unità e integrazione, mentre almeno dell'80% sui test end-to-end$_G$. Sono previsti due incontri in presenza a Padova e due dimostrazioni al reparto R&D.

Tecnologie richieste

- **Vincolanti:** Docker con docker-compose, KNX IoT 3rd-party API, OAuth2, repository$_G$ Git pubblico con licenza open source.
- **Suggerite:** Python con l'SDK FastMCP (alternative TypeScript o Java), AWS Lightsail o EC2, database relazionali o Redis$_G$, CI con GitHub Actions, Terraform, AWS CDK o Ansible.
- **Hardware:** kit fisico portatile (gateway BLE, attuatori, comando tapparelle, sensori, termostato) fornito dall'azienda.

Pro e contro

Pro

- **Requisiti chiari.** Requisiti, casi d'uso e criteri di completamento molto chiari, già classificati per priorità; ampia libertà di partire da un MVP di due o tre casi base.
- **Azienda partecipe.** Azienda preparata, partecipe e disponibile a incontri e a chiarire dubbi, con incontri periodici strutturati.
- **Hardware disponibile.** Hardware reale già disponibile, con un mock generabile dall'OpenAPI e una demo online.
- **Perimetro circoscritto.** Perimetro tecnico circoscritto, con focus prevalentemente sul back-end e nessuno sviluppo di interfacce web o mobile.
- **Tecnologie attuali.** Tecnologie attuali e spendibili (MCP, OAuth2, Infrastructure as Code, AWS, test su più livelli).

Contro

- **Dipendenza infrastrutturale.** Dipendenza da un'infrastruttura proprietaria (cloud Vimar) e un solo kit fisico per sette persone che richiede di organizzare lo sviluppo da remoto tramite i mock e il gateway cloud messi a disposizione.
- **Tecnologie nuove.** Tecnologie quasi tutte nuove per il gruppo (MCP, KNX IoT, OAuth2 con più attori).
- **Documentazione e soglia E2E.** Soglia E2E dell'80% da definire rispetto ai requisiti di progetto; documentazione e infrastruttura da produrre in parallelo allo sviluppo (modello C4, stima dei costi AWS, sito statico).
- **Modelli AI esterni.** Modelli AI esterni non controllabili e costi di utilizzo dei modelli commerciali da chiarire.
- **Interesse parziale.** Dominio non pienamente in linea con gli interessi del gruppo.

Conclusioni

C1 è il capitolato con i criteri di completamento più definiti, un supporto molto strutturato e un risvolto pratico concreto. Il gruppo lo mantiene tra le alternative: le tecnologie sono quasi tutte nuove, la soluzione è quasi interamente back-end, il dominio soddisfa parzialmente gli interessi del gruppo; tuttavia il capitolato C6 tratta un tema (il caching semantico) maggiormente in linea con le preferenze espresse dal gruppo,

mantenendo un perimetro altrettanto chiaro. L'incontro con Vimar del 22/10/2026 ha chiarito i dubbi del gruppo senza modificare il giudizio complessivo (si rimanda al verbale esterno firmato dell'incontro).

C2 - NEXUM Concierge (Eggon)

Descrizione

Eggon S.r.l. sviluppa NEXUM, una piattaforma di comunicazione aziendale e servizi HR composta da un backoffice amministrativo e da una PWA per il personale. Il progetto introduce nella PWA un assistente conversazionale (Concierge) che permette di trovare informazioni, comprendere procedure e accedere ai servizi in linguaggio naturale, collegando la conoscenza aziendale alle funzioni operative della piattaforma. L'MVP è limitato all'area ferie e permessi; combina quattro capacità: regolamenti aziendali caricati dal backoffice, manuali di NEXUM, lettura dei dati personali e registrazione delle richieste con riepilogo e conferma esplicita.

Obiettivo

Il capitolato richiede in un'unica soluzione:

- una chat testuale con gestione del contesto del dialogo;
- un backoffice per caricare, sostituire e ritirare le fonti (PDF testuali), con stato di elaborazione;
- una knowledge base con risposte che citano le fonti e astensione quando le fonti non bastano o si contraddicono;
- abilità di lettura dei dati personali e di registrazione delle richieste;
- autorizzazioni verificate dai servizi, gestione degli errori e degli invii ripetuti, tracciabilità di fonti ed esiti.

Il capitolato definisce 16 requisiti di business (RB-01…RB-16) con criteri di accettazione dettagliati. Non sono richiesti marketplace di plugin, memoria fra conversazioni, voce o allegati.

Tecnologie richieste

Stack NEXUM:

- Angular con PrimeNG (backoffice);
- Angular con Tailwind CSS (PWA);
- Ruby on Rails per le API condivise;
- OAuth2 e JWT;
- PostgreSQL;
- Redis;
- AWS S3;
- Sidekiq.

Lo stack AI e i modelli sono da concordare dopo l'aggiudicazione; l'MVP può essere dimostrato in un ambiente autonomo con servizi di prova o simulatori conformi a un contratto d'integrazione.

Pro e contro

Pro

- Requisiti di business numerati e criteri di accettazione molto specifici; perimetro funzionale definito (ferie e permessi).
- Gruppo di comunicazione dedicato con l'azienda, referente disponibile e percorso strutturato, vicino a un contesto di lavoro reale.
- Regole di sicurezza e tracciabilità chiare.

Contro

- Perimetro ampio e obiettivi che richiedono un'analisi dei requisiti molto accurata per scomporli in attività gestibili.

- Stack AI e modelli da concordare solo dopo l'aggiudicazione; soglie quantitative (accuratezza, tempi, costi) rinviate a una definizione successiva.
- Dipendenza da un contratto di integrazione ancora da stabilire; molte regole da progettare (astensione, conferme, idempotenza).
- Tecnologie poco note al gruppo (Ruby on Rails) e dominio HR di limitato interesse.

Conclusioni

Il gruppo preferisce non procedere alla candidatura per questo capitolato: il perimetro è ampio e richiede un'analisi dei requisiti molto accurata per essere scomposto in attività gestibili, le scelte sullo stack AI sono rinviate a dopo l'aggiudicazione, lo stack di sviluppo è poco noto al gruppo e il dominio ha suscitato limitato interesse; inoltre la componente RAG_G è presente anche in C5, che offre più supporto sul dominio. La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

C3 - FossDot (Uranion)

Descrizione

Uranion EMEA Ltd propone FossDot, uno smart speaker open source, estensibile e rispettoso della privacy, pensato come alternativa self-hosted ad Amazon Echo o Google Nest. Gli smart speaker commerciali sono dispositivi chiusi, legati al cloud del produttore e a un solo assistente, con privacy opaca e obsolescenza programmata. In FossDot il proprietario mantiene il controllo sui dati, sceglie liberamente il provider AI (modelli locali o cloud), collega servizi esterni tramite MCP e abilita Skills e Agents, mentre più dispositivi cooperano in casa.

Obiettivo

Il capitolato richiede quattro componenti:

- scelta e validazione dell'hardware di riferimento (cinque combinazioni proposte e fornite da Uranion), con distinta base e prototipo racchiudibile in un contenitore;
- firmware embedded (Rust preferenziale) per cattura audio, wake word, streaming e riproduzione;
- servizio di bridging STT → LLM → TTS con astrazione sul provider e client MCP;
- interfaccia web per registrare e gestire più dispositivi.

L'MVP richiede il flusso «wake word, domanda, risposta vocale» su un dispositivo fisico. La consegna completa aggiunge più provider (anche locali con $Ollama_G$), MCP, Skills e Agents, multi-dispositivo, aggiornamenti OTA e hardening. Copertura dei test almeno dell'80% per le componenti software, da concordare per il firmware.

Tecnologie richieste

Non è imposto uno stack; è richiesto il rilascio con licenza permissiva (Apache 2.0 o MIT). Riferimenti suggeriti:

- Rust embedded ed Embassy;
- Raspberry Pi o microcontrollori ESP32;
- Seeed ReSpeaker;
- openWakeWord;
- Silero VAD;
- faster-whisper;
- Piper;
- Model Context Protocol;
- Ollama;
- Axum e Tokio;

- containerizzazione e CI/CD$_G$ incoraggiate.

Pro e contro

Pro

- Elevato valore formativo e tecnologie innovative (Rust, MCP, AI locale).
- Referenti dedicati, momenti di approfondimento tecnico e comunicazione chiara.

Contro

- Ambito estremamente ampio, che copre l'intero stack: hardware, firmware real-time, audio, backend asincrono, web.
- Competenze molto eterogenee e in gran parte nuove per il gruppo.
- Reperimento, assemblaggio e calibrazione dell'hardware, con possibili ritardi nelle consegne.
- Latenza della catena vocale e collaudo del firmware difficili da pianificare nelle ore disponibili.

Conclusioni

Il gruppo preferisce non procedere alla candidatura per questo capitolato: pur riconoscendone il valore, l'ampiezza, la varietà delle competenze richieste (dall'hardware al backend) e la gestione dell'hardware fisico comportano un rischio tecnico e organizzativo troppo alto rispetto al totale delle ore e al calendario. La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

C4 - Piattaforma per veicoli e servizi post-vendita (M31 S.r.l.)

Descrizione

M31 S.r.l. propone di sviluppare una piattaforma software distribuita per la gestione del ciclo di vita dei veicoli e dei servizi post-vendita nel settore automotive. Le informazioni su veicoli, manutenzioni e richiami sono oggi frammentate fra sistemi diversi, e questo complica i servizi e la comunicazione con i clienti. La piattaforma vuole permettere la comunicazione tra tre attori principali:

- la casa automobilistica, che gestisce i modelli, i veicoli, la rete di concessionarie e le campagne di richiamo;
- le concessionarie e i centri di assistenza, che gestiscono le vendite, si interfacciano con i proprietari, valutano le richieste di manutenzione e gestiscono gli appuntamenti;
- i proprietari dei veicoli, che possono consultare i dati, fare richieste e ricevere notifiche.

Obiettivo

Il capitolato prevede di centralizzare le informazioni relative a modelli e veicoli, migliorare il flusso di lavoro fra la casa madre e la rete di vendita e assistenza, offrire ai proprietari un unico punto di accesso per gestire le informazioni dell'auto, le manutenzioni, gli appuntamenti, i richiami e le notifiche, con accesso profilato e separato per ruolo.

I prodotti attesi sono un'analisi dello stato dell'arte, un PoC$_G$ che comprenda ad esempio creazione del veicolo, vendita, manutenzione o richiamo, notifica e un MVP con interfacce, API, autenticazione, persistenza e verifica automatizzata.

Tecnologie richieste

L'azienda riporta possibili strumenti per l'implementazione dell'architettura distribuita, quali:

- Node.js e NestJS con TypeScript per lo sviluppo dei servizi applicativi e delle API; Go per componenti che richiedono alte prestazioni o gestione di processi concorrenti;
- NATS o Apache Kafka per la comunicazione asincrona$_G$ e la gestione degli eventi;
- Kubernetes per l'orchestrazione e la gestione centralizzata dei servizi;

- PostgreSQL o MongoDB per la persistenza dei dati;
- Redis per il caching e le sessioni;
- Angular per le interfacce web;
- Prometheus e Grafana per il monitoraggio e le metriche;
- OpenAPI o Swagger per la documentazione delle API.

Pro e contro

Pro

- Dominio concreto e formativo su sistemi distribuiti ed event-driven.
- Lo stack moderno e ampiamente utilizzato nel mondo del lavoro offre un'ottima occasione di crescita formativa.
- Funzionalità e aspettative ampiamente descritte.

Contro

- L'ampiezza del dominio e la ricchezza di funzionalità avanzate aumentano la complessità, con il concreto rischio di generare ritardi nell'analisi dei requisiti.
- Lo sviluppo di un'architettura distribuita a microservizi, basata su broker di messaggistica ed orchestrazione, introduce un alto costo di gestione, di integrazione e di test dei componenti.
- Lo stack di tecnologie proposto è poco conosciuto dal gruppo, che è privo di precedenti esperienze su progetti di tali dimensioni e a cui risulta complesso da padroneggiare nei tempi previsti.

Conclusioni

Il gruppo preferisce non procedere alla candidatura per questo capitolato: il perimetro particolarmente ampio e l'orchestrazione di un sistema a microservizi comportano un onere di analisi, integrazione e test difficile da coniugare con le risorse e il monte ore a disposizione. La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

C5 - RareCare Navigator (Miriade)

Descrizione

Miriade S.r.l. propone RareCare Navigator, una piattaforma digitale con assistente virtuale basato su AI a supporto della gestione di malattie rare e complesse, a partire dalla Sindrome di Sanfilippo. Oggi la rete di cura (centri clinici, pediatrici, terapisti, scuola, servizi sociali) non è coordinata e il carico organizzativo ricade sulla famiglia. Il progetto affianca la figura del Care Navigator (case manager) con un assistente AI, checklist e timeline di cura, un'area di coordinamento con permessi granulari e un fascicolo digitale cifrato. Il prototipo sarà validato con l'associazione Sanfilippo Fighters.

Obiettivo

Il capitolato elenca diciotto requisiti (R1-R18):

- **obbligatori:** permessi e ruoli ($RBAC_G$), workflow e notifiche per scadenze (invalidità, PAI/PEI) via push ed email, chatbot RAG basato solo su fonti ufficiali verificate;
- **opzionali:** assistente vocale, mappatura geospaziale, analisi predittiva dei bisogni;
- **sicurezza e privacy:** conformità GDPR con TLS 1.3 e AES-256, autenticazione a più fattori e OAuth2;
- **architettura e interfaccia:** cloud nativo e $serverless_G$, pattern a plugin per estendere ad altre patologie («Disease Pack», configurazione e non codice), accessibilità WCAG 2.1 AA, frontend cross-platform;
- **servizi AI:** RAG con risposte verificate, estrazione di entità cliniche da testo e voce, checklist dinamiche da PAI/PEI.

Tecnologie richieste

Il capitolato raccomanda:

- Flutter o React Native per il frontend;
- un'architettura serverless su AWS (Lambda, API Gateway) con Amazon Bedrock$_G$, oppure OpenAI o modelli open source.

L'allegato tecnico, non vincolante, propone:

- Cognito (identità e MFA$_G$);
- Amazon Verified Permissions$_G$
- Step Functions ed EventBridge;
- Bedrock Knowledge Bases e Guardrails;
- Aurora Serverless con pgvector;
- S3, SNS e SES, in regione UE.

L'allegato segnala anche le criticità: la cifratura end-to-end in senso stretto è incompatibile con le funzioni AI lato server, l'RBAC da solo non basta, le «allucinazioni» vanno ridotte e misurate con un dataset di valutazione, i dati estratti non vanno scritti senza conferma umana, e servono una DPIA e attenzione a AI Act e dispositivi medici.

Pro e contro

Pro

- **Impatto sociale e formativo.** Forte valore sociale e formativo: AI generativa applicata a un caso reale.
- **Competenze e affiancamento.** Supporto molto strutturato: competenze su cloud, AI, UX$_G$ e dominio clinico, formazione iniziale e possibilità di affiancare una figura specializzata.
- **Flessibilità sul perimetro.** Perimetro negoziabile con il gruppo, requisiti numerati, architettura di riferimento e indicazioni sui flussi dati; una persona del design$_G$ system può aiutare all'inizio inerentemente a struttura e bozza grafica.

Contro

- **Ampiezza del perimetro.** Perimetro molto ampio per il tempo a disposizione: RBAC con relazioni, RAG con astensione, notifiche, fascicolo digitale, app cross-platform.
- **Curva di apprendimento.** Tecnologie quasi tutte nuove per il gruppo (serverless AWS, Bedrock, Flutter): serve tempo di apprendimento.
- **Dati sensibili.** Dati sanitari di minori: GDPR e DPIA, attenzione alla WCAG 2.1 AA, rischio nelle risposte errate dell'AI.
- **Soglie e dataset assenti.** Nessuna soglia di test né dataset di valutazione: va costruito e fatto validare dall'azienda in collaborazione con l'associazione.
- **Incertezze operative e contrattuali.** Account AWS sandbox non ancora garantito, con permessi iniziali limitati; la proprietà del codice a fine progetto è di norma di Miriade (da discutere).

Conclusioni

C5 offre uno dei supporti più ricchi tra i capitolati e un tema di forte valore sociale, ma richiede di costruire quasi da zero un prodotto completo con molte tecnologie nuove, vincoli stringenti su privacy e accessibilità e un dataset di valutazione da creare. Il perimetro è negoziabile, ma il gruppo ritiene più rischiosa la scelta di questo capitolato in rapporto alle ore disponibili rispetto a C6 e C7. Rimane tra le alternative. L'incontro con Miriade del 26/10/2026 ha chiarito i dubbi del gruppo senza modificare il giudizio complessivo (si rimanda al verbale esterno firmato dell'incontro).

C6 - Sistema AI di caching semantico e dashboard FinOps (Var Group)

Descrizione

Var Group usa un'architettura AI a due livelli. Il primo aggrega modelli pubblici (Claude Enterprise, GPT, ricerca web) per attività che non toccano dati riservati. Il secondo, privato su AWS, ospita modelli proprietari, agenti e connettori verso ERP, CRM, SharePoint e Atlassian, con dati classificati su tre livelli di sensibilità (Gold, Silver, Bronze). Un orchestratore AI riceve ogni richiesta, da un processo automatico o da un'interfaccia conversazionale, e la instrada in base all'identità dell'utente e alla natura del dato.

L'instradamento è affidato a LiteLLM$_G$, un proxy$_G$ open source compatibile con le API di OpenAI, già in produzione presso Var Group: gestisce il routing fra più provider, le chiavi e le credenziali. I limiti riscontrati dall'azienda sono:

- cache solo esatta (basata su hash): due domande equivalenti ma formulate in modo diverso generano due chiamate, raddoppiando i costi;
- nessuna dashboard FinOps$_G$ nativa né visibilità aggregata sui costi;
- nessuna previsione (forecast) della spesa;
- multi-tenancy$_G$ limitata, con difficoltà a isolare i costi per team e progetto.

Obiettivo

Estendere LiteLLM con due componenti originali:

- **Cache semantica$_G$:** livello di cache basato su embedding$_G$ e ricerca per similarità, che riconosce le query semanticamente simili e restituisce la risposta già calcolata, se presente. Prevede soglia di similarità configurabile, invalidazione con TTL e versioning, persistenza su Redis o database vettoriale, registrazione di token$_G$ e latenza risparmiati.
- **Dashboard FinOps:** servizio separato che aggrega i consumi per team, progetto e utente, calcola il risparmio reale generato dalla cache, gestisce budget e alert, produce il forecast mensile della spesa, con aggiornamento quasi in tempo reale, multi-tenancy e isolamento dei dati per team e progetto.

Vincoli e criteri: LiteLLM è la base e non va reinventato; copertura dei test almeno del 70% del codice prodotto, con separazione fra test deterministici e valutazione probabilistica; PoC di fattibilità prima dell'implementazione e stand-up periodici.

Prodotti attesi: architettura e Architecture Decision Records, requisiti di business, diagrammi UML, MVP funzionante end-to-end, test con report di copertura e documentazione.

Tecnologie richieste

- **Backend e proxy:** Python, LiteLLM, FastAPI o Flask, Docker.
- **Embedding e similarità:** OpenAI text-embedding-3-small o sentence-transformers (open source); similarità del coseno (numpy, scikit-learn).
- **Cache e vector store$_G$:** Redis (cache con TTL), Qdrant o pgvector, PostgreSQL per le metriche FinOps.
- **Dashboard:** React con Chart.js, oppure Streamlit come alternativa rapida; API REST in JSON.
- **Versionamento:** Git su GitHub$_G$ o Bitbucket.

Rischi e punti aperti

- **Funzioni già presenti in LiteLLM.** Le versioni recenti includono già una cache semantica di base (Redis, Qdrant) e il tracciamento della spesa con budget per team e chiave; non sono previsti forecast. Il valore originale di C6 sta quindi nel potenziare cache e valutazione, nel calcolare il risparmio e il costo complessivo di esercizio, nella storia delle appartenenze ai team, nella previsione di spesa e nella dashboard: va definito con precisione nell'analisi dei requisiti.
- **Cache condivisa.** Var Group desidera una cache unica per tutti: richiede di modificare l'isolamento di default e di valutare le implicazioni sui dati riservati.

- **Falsi hit.** Una risposta sbagliata a una domanda simile ma non equivalente: serve calibrare la soglia con un dataset di coppie di domande.
- **Versione.** Tra la versione in produzione (1.91.2) e quella più recente c'è un divario da considerare.
- **Costi API** per sviluppo e test di embedding e modelli, mitigati dall'ambiente con budget fornito da Var Group.

Motivazione della scelta

Il gruppo ha scelto C6 come prima scelta per i seguenti motivi:

- **Tema attuale e rilevante.** Rendere sostenibile l'AI monitorandone i costi è un problema reale, comune a molti altri progetti, e il risultato è misurabile in token e costi risparmiati, latenza e accuratezza della cache. L'MVP si può validare numericamente e dà contenuto concreto al Piano di Qualifica.
- **Perimetro ben definito.** Due componenti su un proxy esistente, senza hardware e senza necessità di gestire i dati sensibili, essendo già in ambiente a basso rischio.
- **Criteri di qualità chiari.** La soglia di copertura (70%) è esplicita e raggiungibile, e la distinzione fra test deterministici e valutazione probabilistica è un tema formativo importante.
- **Supporto strutturato.** Design thinking, formazioni mirate su LiteLLM (2 ore), Embedding e Caching (2 ore) e FinOps con dashboard (1,5 ore), revisioni e incontri ogni due settimane.
- **Competenze di partenza.** Python, Docker, SQL e Git sono già noti al gruppo e alcuni componenti conoscono le API REST: le tecnologie sono in buona parte note, diversamente da altri capitolati.
- **Fattibilità.** Un PoC per la RTB_G (cache semantica minimale e aggregazione dei costi) e un MVP validabile nei tempi del progetto sono realistici.

Conclusioni

Il gruppo si candida ufficialmente per lo sviluppo del capitolato C6: il tema è attuale, il problema reale e misurabile, il perimetro è chiaro dopo l'incontro con Var Group, le tecnologie sono in buona parte note e il supporto del proponente è strutturato. Il valore del progetto sta nel portare cache semantica e FinOps di LiteLLM al livello richiesto, partendo da ciò che LiteLLM già offre. Il gruppo è motivato e ritiene realizzabile il lavoro nei tempi e con i vincoli del progetto. L'incontro con Var Group del 26/10/2026 (si rimanda al verbale esterno firmato) ha chiarito i dubbi del gruppo.

C7 - SmartDashboard (Ergon Informatica)

Descrizione

Ergon Informatica S.r.l. propone SmartDashboard, una piattaforma di Business Intelligence end-to-end destinata alle aziende dei settori alimentare e dei trasporti. Nel gestionale ERP queste aziende accumulano dati preziosi (ordini, vendite, magazzino, consegne, marginalità) in una forma pensata per l'operatività e non per l'analisi: ogni nuovo report passa dal reparto IT e i numeri grezzi non spiegano il perché degli andamenti. La piattaforma recupera i dati dall'ERP, li consolida in un data $warehouse_G$ ottimizzato per l'analisi ed espone una web app in cui l'utente costruisce in autonomia dashboard interattive. Ciò che la distingue da un comune strumento di reportistica è un livello di intelligenza artificiale con due finalità: abbattere la barriera tecnica, generando un grafico da una richiesta in linguaggio naturale, e arricchire l'analisi con l'interpretazione automatica dei dati (anomalie, tendenze, spiegazioni).

Obiettivo

Il capitolato prevede un'architettura a tre livelli: un *Raw Data Store* con copia storicizzata dei dati grezzi, un *Data Warehouse* con ETL_G e modello dimensionale a star schema, e una web app con backend che espone i dati tramite API (senza rendere accessibili al frontend le interrogazioni dirette) e un costruttore di dashboard.

- **Requisiti funzionali$_G$:** dashboard drag and drop su griglia reattiva con configurazioni per utente; libreria di widget (barre, linee, torta, KPI, gauge, tabelle, pivot, mappe); filtri trasversali; temi chiaro e scuro; esportazione in PDF ed Excel; autenticazione.
- **Layer AI, in ordine di priorità:** generazione di visualizzazioni da linguaggio naturale (capacità centrale), insight automatici, assistente designer, spiegazione di un indicatore su richiesta (estensione). L'AI opera solo su dati e metadati esposti dalla piattaforma, con controlli di coerenza delle risposte.
- **Fasi consigliate:** prima la BI funzionante, poi la generazione da linguaggio naturale, poi insight e assistente, infine la spiegazione. Il capitolato non fissa soglie numeriche di test.

Tecnologie richieste

Le tecnologie sono suggerite e non vincolanti:

- **Data warehouse:** DuckDB (alternative ClickHouse, MonetDB, Apache DataFusion); sorgente dati sostituibile come requisito desiderabile.
- **Raw store e configurazioni:** PostgreSQL, MySQL o MariaDB.
- **ETL:** Python con Polars o Pandas, eventualmente con orchestratori (Airflow, Prefect, Dagster); connettori ODBC o middleware$_G$ JSON verso l'ERP.
- **AI:** LLM tramite API commerciale (OpenAI, Anthropic); LangChain o Vanna.ai per la traduzione da linguaggio naturale a dati.
- **Frontend:** React, Angular o Vue con ECharts o Chart.js, AG Grid per tabelle e pivot; comunicazione con il backend via API REST.

Pro e contro

Pro

- **Percorso formativo completo.** Percorso completo e formativo del dato: ETL, modellazione dimensionale, API, frontend e AI su dati reali.
- **Supporto concreto.** Supporto concreto: dataset di test, chiavi API con credito, dati ERP, indicazioni su whitelist e test dell'LLM, allineamento ricorrente ogni quindici giorni.
- **Fasi consigliate.** Fasi consigliate che permettono di avere una BI funzionante prima di aggiungere l'AI.
- **Stack libero e noto.** Stack lasciato libero e ben suggerito; database e frontend già conosciuti dal gruppo.

Contro

- **Ampiezza del perimetro.** Perimetro molto ampio, con priorità da stabilire durante l'analisi dei requisiti per non superare le ore disponibili.
- **Soglie di test assenti.** Nessuna soglia numerica per i test.
- **Rischio di inaccuratezza.** Affidabilità del layer AI difficile da garantire (join errati, doppi conteggi, ambiguità fra valori lordi e netti), con test da rieseguire a ogni cambio di modello.
- **Complessità del layer AI.** Struttura del layer AI complessa (analisi delle tabelle e generazione delle query); tecnologie in parte poco note al gruppo.
- **Aggiornamento dati da definire.** Dati ERP statici nella fase iniziale: l'aggiornamento periodico resta da definire.

Conclusioni

C7 offre un percorso completo e seguito, risposte concrete e materiali di prova forniti dall'azienda: risulta quindi la seconda scelta del gruppo. Restano un perimetro dei requisiti molto ampio da circoscrivere, la mancanza di soglie numeriche di test, la difficoltà di garantire l'affidabilità del layer AI e un tema di interesse medio per il gruppo. L'incontro con Ergon Informatica del 22/10/2026 (si rimanda al verbale esterno firmato) ha chiarito i dubbi del gruppo, in particolare sulla fornitura dei dati ERP.

C8 - Sonar (Halue)

Descrizione

Halue S.r.l. Società Benefit propone Sonar, un'applicazione web che si collega a uno store Shopify e ne ricostruisce la mappa: cosa esiste, cosa dipende da cosa e quali componenti non sono più utilizzati. Con il tempo gli store di grandi dimensioni si stratificano accumulando app installate e dimenticate, webhook$_G$ verso fornitori non più attivi, campi personalizzati mai letti, script e funzioni di cui si è persa la traccia e codice di tema non più referenziato. Questa conoscenza non è documentata e si perde al cambio di fornitore o di personale, allungando i tempi di intervento e aumentando i rischi di incidenti in produzione.

Obiettivo

Il capitolato richiede lo sviluppo di un sistema articolato su cinque capacità chiave:

- scansione durevole e rispettosa dei limiti di utilizzo delle Admin API, con snapshot immutabili e datati;
- normalizzazione in un grafo di dipendenze in cui ogni arco riporta fonte e livello di confidenza;
- analisi statica del codice Liquid del tema e misurazione dell'uso effettivo dei metafield;
- motore di regole estensibile (almeno otto regole) per orfani e codice morto;
- interfaccia di navigazione, confronto fra snapshot ed esportazione della documentazione in Markdown o HTML.

Sono definiti 20 requisiti funzionali obbligatori, 7 di qualità e sicurezza e 7 non funzionali. Soglie di accettazione: test superati almeno al 95%, falsi positivi ad alta severità al massimo del 10%, copertura almeno del 70%.

Tecnologie richieste

Obbligatorie:

- Next.js (App Router) su Vercel;
- Vercel Workflows;
- Vercel Cron Jobs;
- Supabase (PostgreSQL) con Row Level Security;
- Supabase Auth con solo provider Google;
- TypeScript strict;
- Shopify Admin GraphQL API tramite custom app;
- Git e GitHub con CI.

Fortemente consigliate:

- Vercel Blob, Queues, Sandbox e AI Gateway;
- Drizzle o Prisma;
- Tailwind e shadcn/ui;
- React Flow o Cytoscape.js;
- Vitest;
- Playwright;
- MSW.

Pro e contro

Pro

- Documentazione precisa e accurata: requisiti, metriche di qualità e criteri di accettazione definiti in modo oggettivo e misurabile; indicazioni utili sull'ambiente di lavoro e sulla flessibilità in caso di limiti dei piani gratuiti.
- Ottimi spunti di architettura (snapshot immutabili, workflow durevoli, confidenza esplicita) e politica chiara sull'uso dell'AI nello sviluppo.

Contro

- Stack imposto con tecnologie molto specifiche (Workflows durevoli, Row Level Security) e limiti dei piani gratuiti.
- Lo store Shopify di test non viene fornito e deve essere costruito dal gruppo per costituire l'ambiente di prova.
- Dominio specifico (Shopify) e supporto limitato a videocall bisettimanali ed e-mail, senza incontri in presenza.

Conclusioni

Il gruppo preferisce non procedere alla candidatura per questo capitolato: ne apprezza la chiarezza, ma lo stack vincolante ed il perimetro già fissato lo rendono poco compatibile con le ore a disposizione, e il gruppo preferisce capitolati non legati a una particolare piattaforma di hosting o a un dominio specifico. La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

C9 - SpecCheck (Zucchetti)

Descrizione

Zucchetti S.p.A. parte da una constatazione: con i Large Language Model il collo di bottiglia non è più scrivere codice ma verificarlo. Compilatori, linter e test dicono se il codice è corretto, non se rispetta l'intento che lo ha motivato. Il capitolato richiama il «vibe coding» (sviluppare descrivendo le intenzioni in linguaggio naturale) e spiega perché oggi contano gli harness, cioè i sistemi che gestiscono contesto e scelte di progetto (Claude Code, Codex, OpenCode e altri) tramite file guida come AGENTS.md. Gli harness si estendono con skill, comandi, tool e plugin.

Zucchetti, interessata a verificare l'estendibilità di OpenCode, propone di sviluppare SpecCheck: un insieme coordinato di skill, comandi, tool e almeno un plugin che, su un progetto software esterno, verifichi la tracciabilità casi d'uso → requisiti → codice. Il sistema verifica e non realizza.

Obiettivo

Il capitolato richiede nove compiti obbligatori:

- leggere i diagrammi dei casi d'uso e la documentazione aggiuntiva;
- leggere le specifiche dei requisiti e confrontarle con i casi d'uso, segnalando le mancate corrispondenze;
- leggere il codice, capirne il comportamento e verificare che implementi quanto richiesto;
- mostrare in una dashboard (anche testuale) casi d'uso rilevati nei requisiti e requisiti rilevati nel codice, con le discrepanze ben visibili.

Il confronto non può essere affidato in modo deterministico a un LLM: va realizzato come plugin per OpenCode o applicativo esterno richiamato da skill e comandi. Il prodotto è un pacchetto unico con script di installazione (Bash o PowerShell). Sono opzionali la dashboard grafica, la copertura inversa del codice rispetto alle specifiche e la lettura da PDF.

Tecnologie richieste

- Harness: OpenCode, con skill, comandi, tool (script deterministici, ad esempio JavaScript/Node) e plugin.
- Formati: diagrammi in Mermaid o XMI, specifiche in Markdown; PDF opzionale.
- Riferimenti suggeriti: OpenSpec, Spec-kit, BMAD, PlantUML, UMLet.
- Modelli: LLM di grandi dimensioni con tetto di costo, modelli piccoli (ad esempio Gemma 4 26B, Qwen 27B) senza limiti.
- Distribuzione: repository pubblico (titolo preferenziale) e script di installazione.

Pro e contro

Pro

- Tema vicino al corso: la tracciabilità tra casi d'uso, requisiti e codice è un argomento centrale dell'ingegneria del software.
- Argomento attuale (sviluppo assistito da AI) con strumenti open source e nessuna dipendenza hardware.
- Dashboard testuale accettata, quindi maggior possibilità di concentrarsi sulle funzionalità.
- Modelli piccoli senza limiti di utilizzo per le prove.

Contro

- Progetto fortemente esplorativo: l'esito dipende dalla capacità degli LLM di comprendere codice e specifiche, e il capitolato ammette che alcune parti potrebbero non essere definite con certezza.
- Nessuna soglia di accettazione né benchmark: non è definito come valutare una «verifica corretta», e il confronto semantico è probabilistico, con falsi positivi e negativi difficili da misurare.
- Manca un progetto di prova con discrepanze note, consigliato ma non fornito.
- Competenze nuove per il gruppo (estensione di OpenCode, analisi del codice, parsing di Mermaid e XMI).

Conclusioni

Il gruppo riconosce l'interesse del tema e la sua pertinenza con il corso, ma preferisce non procedere alla candidatura: mancano criteri di accettazione misurabili, l'esito dipende in larga parte dagli LLM, per questo dimostrare un MVP convincente sarebbe più rischioso che in C6, dove il risultato si misura in numeri (token e costi risparmiati, latenza, accuratezza). La valutazione si basa unicamente sulla documentazione fornita e sulla presentazione svolta in aula, non essendo stato svolto alcun incontro con l'azienda.

Appendice A - Contesto e vincoli del gruppo

- **Composizione:** sette persone: Matteo Arcandi, Jacopo De Donà, Jacopo Franco, Matteo Marcon, Riccardo Peresson, Paolo Prevedello Ramirez, Chiara Protopapa.
- **Impegno:** ore produttive$_G$ a persona entro l'intervallo 80-95 previsto dal Regolamento, come indicato nella Dichiarazione degli Impegni.
- **Calendario:** circa 20 settimane dall'aggiudicazione del 3 novembre 2026, con rallentamenti per la pausa natalizia e la sessione d'esame invernale; data di consegna come indicato nella Dichiarazione degli Impegni.
- **Costo:** preventivo non inferiore a 12.000 €, il minimo previsto per un gruppo di sette persone.
- **Revisioni:** RTB (requisiti e proof of concept) e PB$_G$ (specifica tecnica e MVP validato dal proponente).

Appendice B - Incontri esplorativi

Capitolato	Proponente	Data	Partecipanti	Verbale
C7 SmartDashboard	Ergon Informatica	22/10/2026, 15:30-15:50	Gianluca Carlesso (Ergon); Paolo Prevedello Ramirez, Chiara Protopapa, Jacopo Franco	Verbale esterno nel repository documentale
C1 AURA	Vimar	22/10/2026, 16:00-17:00	Mariano Sciacco e Domenico Bevilacqua (Vimar); Jacopo De Donà e Matteo Marcon	Verbale esterno nel repository documentale

C6 Caching semantico e FinOps	Var Group	26/10/2026, 15:00-15:40	Stefano Dindo, Laura Ferretti, Alessio Borgheresi, Michele Massaro (Var Group); Chiara Protopapa, Jacopo Franco, Jacopo De Donà e Matteo Marcon	Verbale esterno nel repository documentale
C5 RareCare Navigator	Miriade	26/10/2026, 17:00-17:30	Emanuele Righetto (Miriade); Chiara Protopapa, Riccardo Peresson, Jacopo De Donà e Matteo Marcon	Verbale esterno nel repository documentale

Per ogni incontro il gruppo ha raccolto le domande in modo asincrono in un documento condiviso e ha chiesto il permesso di registrare e trascrivere la riunione. I verbali, con decisioni e attività, sono pubblicati nel repository documentale e nel sito del gruppo.

Paolo Prevedello Ramirez

Firma del Responsabile di Progetto